Modelo de Amenazas PASTA — Cliente A

> **Nota sobre anonimización.** Este informe documenta un trabajo real, realizado con autorización del propietario. El nombre del cliente y los identificadores de su infraestructura se reemplazaron por marcadores (`Cliente A` , `cliente-a`) antes de publicarlo. Belona Cybersecurity no publica datos identificables de un cliente sin su autorización por escrito.

Stages 3 a 7 (las 1-2, Objetivos y Alcance, se documentaron previamente)

PASTA = Process for Attack Simulation and Threat Analysis. Metodología de modelado de amenazas centrada en el riesgo, en 7 etapas. Este documento cubre las etapas 3-7 aplicadas al sistema (web + Firebase).

Stage 3 — Descomposición de la Aplicación (ADA)

Componentes

Componente	Tecnología	Rol
Frontend público	HTML/CSS/JS (Firebase Hosting)	Sitio + formulario de contacto
Panel admin	admin.html	Gestión (guías, blog, consultas, procesos, usuarios, logs)
Autenticación	Firebase Auth + Identity Platform	Login + MFA TOTP
Base de datos	Firestore (`nam5`)	guias, servicios, posts, consultas, procesos_insolvencia, auditoria
Archivos	Firebase Storage	Documentos de expedientes
Backend	Cloud Functions (Node 24)	enviarConsulta, admin* , auditarSituacion

Actores / Roles

- **Público anónimo** (visitantes del sitio; envían consultas)
- **editor** (edita blog), **abogado** (servicios/consultas/procesos), **admin** (todo + usuarios)
- **Atacante externo**, **insider malicioso** (rol de bajo privilegio)

Activos a proteger

- Datos personales de clientes (consultas, expedientes) — **sensibles, Ley 1581**
- Documentos legales (Storage)
- Credenciales y secretos
- Integridad y disponibilidad del sitio

Límites de confianza (trust boundaries)

1. Navegador ↔ Firebase (Internet). 2. Cliente ↔ reglas Firestore/Storage (server-side).
2. Cliente ↔ Cloud Functions (App Check + auth). 4. Repo/CI ↔ despliegue.

Puntos de entrada

Formulario de contacto · Login del panel · Llamadas a Cloud Functions · Lectura pública de contenido · Pipeline CI/CD.

Stage 4 — Análisis de Amenazas (TA)

Fuentes de amenaza: atacante externo oportunista/dirigido, bots, insider de bajo privilegio, error humano, cadena de suministro.

Amenazas principales (estilo STRIDE + contexto real): 1. **Acceso no autorizado a datos** de clientes (Information Disclosure) 2. **Escalada de privilegios** (Elevation of Privilege) 3. **Inyección / XSS** en datos de origen público (Tampering) 4. **Robo de credenciales** / secuestro de cuenta (Spoofing) 5. **Fuga de código/secretos** (Information Disclosure — cadena de despliegue) 6. **Abuso del formulario / DoS de costos** (Denial of Service) 7. **Manipulación de archivos servidos** (Tampering — defacement) 8. **Cadena de suministro** (dependencias vulnerables)

Stage 5 — Análisis de Vulnerabilidades y Debilidades (WVA)

Debilidad	CWE/OWASP	Estado
Exposición de `.git` + token (config de deploy)	CWE-538 / OWASP A05	**Hallada en pentest, corregida**
Código de servidor expuesto en Hosting	CWE-540	Corregida
Faltaban cabeceras de seguridad (clickjacking, MIME)	OWASP A05	Corregida
Dependencias vulnerables (functions)	OWASP A06	Mitigada (sin high/critical) + Dependabot + audit en CI
Scripts inline sin CSP estricta	OWASP A05	Aceptada (riesgo bajo; CSP frame-ancestors puesta)
Endpoint público de contacto (abuso)	OWASP A04	Mitigada (App Check + validación server-side)
XSS almacenado en consultas/procesos	OWASP A03	Mitigada (escape `esc()` en el panel)

Stage 6 — Modelado de Ataques (AM)

Árboles de ataque de las amenazas top (y el control que corta cada camino):

Objetivo del atacante: leer datos de clientes - Vía 1: leer Firestore directo sin login → **BLOQUEADO** por reglas (validado: permission-denied) - Vía 2: loguearse como admin → requiere credencial **+ MFA** (mitigado) - Vía 3: usuario de bajo privilegio (editor) lee consultas → **BLOQUEADO** por reglas (validado) - Vía 4: bajar el repo por `.git` → estaba **ABIERTO**, ahora **CERRADO**

Objetivo: convertirse en admin (escalada) - Llamar `adminAsignarRol` como editor → **BLOQUEADO** (App Check + `exigirAdmin`, validado)

Objetivo: manipular el sitio (defacement) - Alterar archivos servidos → detectado por **FIM** (validado con alerta Telegram)

Objetivo: robar documentos de expedientes - Acceder a Storage sin rol → **BLOQUEADO** (reglas `storage/unauthorized`, validado)

Stage 7 — Análisis de Riesgo e Impacto (RIA)

Riesgo = Probabilidad × Impacto. Impacto alto por ser datos legales/sensibles (Ley 1581).

Amenaza	Prob. (con controles)	Impacto	Riesgo residual
Acceso no autorizado a datos	Baja	Alto	**Bajo**
Escalada de privilegios	Baja	Alto	**Bajo**
XSS almacenado	Baja	Medio	**Bajo**
Robo de credenciales	Baja (MFA)	Alto	**Bajo-Medio**
Fuga de código/secretos	Baja (corregida)	Alto	**Bajo**
Abuso/DoS de costos	Media	Bajo	**Bajo** (App Check + alerta presupuesto)
Defacement	Baja	Medio	**Bajo** (FIM detecta)
Dependencias vulnerables	Baja	Medio	**Bajo** (Dependabot + audit CI)

Contramedidas implementadas

Prevención: RBAC (admin/abogado/editor), reglas Firestore/Storage, App Check, MFA, validación server-side, escape anti-XSS, cabeceras de seguridad, gestión de secretos. Detección: FIM + Logs de auditoría. Recuperación: backups (PITR + diarios + offsite 3-2-1) + runbooks. Cumplimiento: consentimiento con prueba, política 1581 con transferencia internacional.

Recomendaciones (riesgo aceptado / futuro)

1. CSP estricta de scripts (refactor inline). 2. Validación de campos en reglas (tamaño/tipo).

2. Migrar GEMINI_KEY a Secret Manager. 4. Alerta de presupuesto. 5. Repetir pentest tras cambios mayores.

Conclusión

El sistema presenta **riesgo residual bajo** en las amenazas relevantes, con controles en las tres capas (prevención, detección, recuperación) validados mediante pentest autorizado. Modelo a revisar ante cambios significativos.
